T4_PRACTICE_PROGRAM

December 15, 2025

1 write a below python program for cricket analysis IN STREAMLIT

Class Data:

```
The Data class is an abstract base class (ABC) that defines a common interface for handling spo

    Class Variables: .Array (Class Variable):
```

A class-level numpy array that holds the initial sports data.It represents scores of players in various matches, organized as a 2D arrayThere is an array of scores of 6 Batsmen in 5 T20 Matches. Which is given below. Array=np.array([[64, 13, 100, 46, 69, 85], [54, 43, 7, 34, 17, 19], [73, 73, 90, 77, 53, 14], [74, 41, 41, 83, 74, 95], [48, 31, 74, 49, 67, 99]])

Abstract Method: abcmethod(self) (Abstract Method)

```
Class Match (Child/Sub class of Data):

    The Match class inherits from the abstract class Data and provides concrete implementations

    1.match_highscore(self, match_number):
Finds and prints the maximum score in a specific match specified by match_number.

2.player_lowscore(self, player_number):
Determines and prints the minimum score of a specific player across all matches.

3.match_total(self):
```

Adds an extra column to the existing array to represent the total score of each player across all matches. (Add the last column as the total scores column to the existing array and print the updated array.)

4.sorted_matchtotal_hightolow(self): Sorts the array based on the total scores of players across all matches in descending order. Utilizes the match_total method to compute the total scores for each player. prints the sorted array.

5. player total score(self): print the total scores of player

6.binary_array(self): Create binary array if score>50 denote 1 or score<50 denote 2

Vishal Acharya

7.lineplot (self, player1, player2): Generates a line plot showing the scores of two players across all matches. Retrieves the scores of the specified players from the array and plots them against the match numbers.

8. pieplot(self, match_number): Generates a pie chart showing the distribution of scores for a specific match.

[1]:
```
%%writefile cricket.py
import streamlit as st
import numpy as np
import matplotlib.pyplot as plt
from abc import ABC, abstractmethod

# ================= ABSTRACT BASE CLASS =================
class Data(ABC):
    # Class variable: Scores of 6 batsmen in 5 matches
    Array = np.array([
        [64, 13, 100, 46, 69, 85],
        [54, 43, 7, 34, 17, 19],
        [73, 73, 90, 77, 53, 14],
        [74, 41, 41, 83, 74, 95],
        [48, 31, 74, 49, 67, 99]
    ])

    @abstractmethod
    def abcmethod(self):
        pass


# ================= CHILD CLASS =================
class Match(Data):

    def abcmethod(self):
        return "Cricket Match Data Analysis"

    # 1. Highest score in a match
    def match_highscore(self, match_number):
        return np.max(self.Array[match_number - 1])

    # 2. Lowest score of a player
    def player_lowscore(self, player_number):
        return np.min(self.Array[:, player_number - 1])

    # 3. Add total column
    def match_total(self):
        totals = np.sum(self.Array, axis=1).reshape(-1, 1)
        return np.hstack((self.Array, totals))
```

Vishal Acharya

```python
    # 4. Sort by total score (high to low)
    def sorted_matchtotal_hightolow(self):
        arr = self.match_total()
        return arr[arr[:, -1].argsort()[::-1]]

    # 5. Player total score
    def player_total_score(self):
        return np.sum(self.Array, axis=0)

    # 6. Binary array
    def binary_array(self):
        return np.where(self.Array > 50, 1, 2)

    # 7. Line plot
    def lineplot(self, player1, player2):
        fig, ax = plt.subplots()
        ax.plot(self.Array[:, player1 - 1], marker='o', label=f'Player␣
↪{player1}')
        ax.plot(self.Array[:, player2 - 1], marker='o', label=f'Player␣
↪{player2}')
        ax.set_xlabel("Matches")
        ax.set_ylabel("Scores")
        ax.set_title("Player Performance Comparison")
        ax.legend()
        return fig

    # 8. Pie plot
    def pieplot(self, match_number):
        fig, ax = plt.subplots()
        ax.pie(self.Array[match_number - 1],
               labels=[f'Player {i+1}' for i in range(6)],
               autopct='%1.1f%%')
        ax.set_title(f"Match {match_number} Score Distribution")
        return fig


# ================= STREAMLIT UI =================
st.set_page_config(page_title="Cricket Analysis", layout="wide")
st.title("  Cricket Match Analysis Using Streamlit")

match = Match()

st.subheader("  Original Score Array")
st.write(match.Array)

menu = st.sidebar.selectbox(
    "Choose Analysis",
```

Vishal Acharya

```
    [
        "Match High Score",
        "Player Low Score",
        "Add Match Total Column",
        "Sort by Total Score",
        "Player Total Scores",
        "Binary Score Array",
        "Line Plot (Player Comparison)",
        "Pie Plot (Match Distribution)"
    ]
)

# ---------------- MENU ACTIONS ----------------
if menu == "Match High Score":
    m = st.number_input("Enter Match Number (1-5)", 1, 5)
    st.success(f"Highest Score in Match {m}: {match.match_highscore(m)}")

elif menu == "Player Low Score":
    p = st.number_input("Enter Player Number (1-6)", 1, 6)
    st.success(f"Lowest Score of Player {p}: {match.player_lowscore(p)}")

elif menu == "Add Match Total Column":
    st.subheader("Array with Total Scores Column")
    st.write(match.match_total())

elif menu == "Sort by Total Score":
    st.subheader("Sorted by Total Score (High → Low)")
    st.write(match.sorted_matchtotal_hightolow())

elif menu == "Player Total Scores":
    totals = match.player_total_score()
    for i, score in enumerate(totals, start=1):
        st.write(f"Player {i} Total Score: {score}")

elif menu == "Binary Score Array":
    st.subheader("Binary Array ( >50 → 1,  50 → 2 )")
    st.write(match.binary_array())

elif menu == "Line Plot (Player Comparison)":
    p1 = st.number_input("Player 1 (1-6)", 1, 6, 1)
    p2 = st.number_input("Player 2 (1-6)", 1, 6, 2)
    fig = match.lineplot(p1, p2)
    st.pyplot(fig)

elif menu == "Pie Plot (Match Distribution)":
    m = st.number_input("Match Number (1-5)", 1, 5)
    fig = match.pieplot(m)
```

Vishal Acharya

```
    st.pyplot(fig)

st.markdown("---")
st.caption("Developed for Cricket Data Analysis using OOP + Streamlit")
```

```
Writing cricket.py
```

2 Problem Definition

Design a Student Marks Analysis System to analyze the performance of students across multiple subjects using Object-Oriented Programming, NumPy, and Streamlit.

The system stores marks of 5 students in 6 subjects using a 2D NumPy array.

Given Data (Class Variable)

Rows → Students Columns → Subjects

Array = np.array([[78, 65, 89, 72, 90, 84], [56, 62, 71, 68, 60, 66], [88, 91, 94, 90, 86, 92], [45, 52, 58, 49, 55, 50], [70, 75, 80, 78, 82, 77]])

Class Structure Abstract Base Class: AcademicData

Class Variable

Array → NumPy marks data

Abstract Method

abcmethod(self) → system description

Child Class: StudentAnalysis(AcademicData)

Implements student-wise and subject-wise analysis.

Required Methods

student_highscore(self, student_no) → Highest marks of a student

subject_lowscore(self, subject_no) → Lowest marks in a subject

student_total(self) → Add total marks column

sorted_total_hightolow(self) → Sort students by total marks (descending)

student_average(self) → Average marks of each student

grade_array(self) → Marks 60 → "Pass" else "Fail"

lineplot(self, student1, student2) → Compare two students' performance

barplot(self) → Bar chart of total marks

[3]:
```
%%writefile student.py
import streamlit as st
import numpy as np
import matplotlib.pyplot as plt
```

Vishal Acharya

```python
from abc import ABC, abstractmethod

# ========== ABSTRACT CLASS ==========
class AcademicData(ABC):

    Array = np.array([
        [78, 65, 89, 72, 90, 84],
        [56, 62, 71, 68, 60, 66],
        [88, 91, 94, 90, 86, 92],
        [45, 52, 58, 49, 55, 50],
        [70, 75, 80, 78, 82, 77]
    ])

    @abstractmethod
    def abcmethod(self):
        pass


# ========== CHILD CLASS ==========
class StudentAnalysis(AcademicData):

    def abcmethod(self):
        return "Student Marks Analysis System"

    def student_highscore(self, student_no):
        return np.max(self.Array[student_no - 1])

    def subject_lowscore(self, subject_no):
        return np.min(self.Array[:, subject_no - 1])

    def student_total(self):
        totals = np.sum(self.Array, axis=1).reshape(-1, 1)
        return np.hstack((self.Array, totals))

    def sorted_total_hightolow(self):
        arr = self.student_total()
        return arr[arr[:, -1].argsort()[::-1]]

    def student_average(self):
        return np.mean(self.Array, axis=1)

    def grade_array(self):
        return np.where(self.Array >= 60, "Pass", "Fail")

    def lineplot(self, s1, s2):
        fig, ax = plt.subplots()
        ax.plot(self.Array[s1-1], marker='o', label=f'Student {s1}')
```

Vishal Acharya

```python
        ax.plot(self.Array[s2-1], marker='o', label=f'Student {s2}')
        ax.set_xlabel("Subjects")
        ax.set_ylabel("Marks")
        ax.set_title("Student Performance Comparison")
        ax.legend()
        return fig

    def barplot(self):
        totals = np.sum(self.Array, axis=1)
        fig, ax = plt.subplots()
        ax.bar([f'S{i+1}' for i in range(len(totals))], totals)
        ax.set_title("Total Marks of Students")
        return fig


# ========== STREAMLIT APP ==========
st.set_page_config(page_title="Student Analysis", layout="wide")
st.title("  Student Marks Analysis System")

obj = StudentAnalysis()

st.subheader("Marks Data")
st.write(obj.Array)

menu = st.sidebar.selectbox(
    "Select Operation",
    [
        "Student High Score",
        "Subject Low Score",
        "Add Total Column",
        "Sort by Total Marks",
        "Student Average",
        "Pass/Fail Array",
        "Line Plot Comparison",
        "Bar Plot Totals"
    ]
)

if menu == "Student High Score":
    s = st.number_input("Student Number (1-5)", 1, 5)
    st.success(obj.student_highscore(s))

elif menu == "Subject Low Score":
    sub = st.number_input("Subject Number (1-6)", 1, 6)
    st.success(obj.subject_lowscore(sub))

elif menu == "Add Total Column":
```

Vishal Acharya

```
    st.write(obj.student_total())

elif menu == "Sort by Total Marks":
    st.write(obj.sorted_total_hightolow())

elif menu == "Student Average":
    avg = obj.student_average()
    for i, v in enumerate(avg, 1):
        st.write(f"Student {i} Average: {v:.2f}")

elif menu == "Pass/Fail Array":
    st.write(obj.grade_array())

elif menu == "Line Plot Comparison":
    s1 = st.number_input("Student 1", 1, 5, 1)
    s2 = st.number_input("Student 2", 1, 5, 2)
    st.pyplot(obj.lineplot(s1, s2))

elif menu == "Bar Plot Totals":
    st.pyplot(obj.barplot())
```

```
Writing student.py
```

3 Problem Theme: University Examination Result Processing System

Description (for students)

A university conducts exams for multiple departments. Each student has:

Roll number

Name

Marks in 5 subjects

Rules:

Passing marks per subject = 40

Total subjects = 5

Result depends on individual subject marks and overall percentage

Grade system applies only if the student passes all subjects

You must use inheritance to extend functionality step by step.

[1]:
```
# Base class
class Student:
    university = "National Technical University"   # class variable
    total_subjects = 5                              # class variable
```

Vishal Acharya

```python
    def __init__(self, roll, name):
        self.roll = roll
        self.name = name

    def display_student(self):
        print("University:", Student.university)
        print("Roll No:", self.roll)
        print("Name:", self.name)


# Derived class 1
class Exam(Student):
    pass_mark = 40    # class variable

    def __init__(self, roll, name, marks):
        Student.__init__(self, roll, name)
        self.marks = marks

    def check_pass_fail(self):
        for m in self.marks:
            if m < Exam.pass_mark:
                return False
        return True


# Derived class 2
class Result(Exam):
    max_marks = 500   # class variable

    def __init__(self, roll, name, marks):
        Exam.__init__(self, roll, name, marks)

    def calculate_total(self):
        total = 0
        for m in self.marks:
            total = total + m
        return total

    def calculate_percentage(self):
        return self.calculate_total() * 100 / Result.max_marks

    def calculate_grade(self):
        per = self.calculate_percentage()
        if per >= 75:
            return "A"
        elif per >= 60:
```

Vishal Acharya

```python
            return "B"
        elif per >= 50:
            return "C"
        else:
            return "D"

    def display_result(self):
        self.display_student()
        print("Marks:", self.marks)
        print("Total Marks:", self.calculate_total())
        print("Percentage:", self.calculate_percentage())

        if self.check_pass_fail():
            print("Result: PASS")
            print("Grade:", self.calculate_grade())
        else:
            print("Result: FAIL")
            print("Grade: Not Applicable")


# -------- Object Creation --------
student1 = Result(101, "Amit", [78, 69, 85, 74, 90])
student1.display_result()

print("\n----------------------\n")

student2 = Result(102, "Ravi", [45, 32, 60, 55, 48])
student2.display_result()
```

```
University: National Technical University
Roll No: 101
Name: Amit
Marks: [78, 69, 85, 74, 90]
Total Marks: 396
Percentage: 79.2
Result: PASS
Grade: A

----------------------

University: National Technical University
Roll No: 102
Name: Ravi
Marks: [45, 32, 60, 55, 48]
Total Marks: 240
Percentage: 48.0
Result: FAIL
Grade: Not Applicable
```

Vishal Acharya

4 Problem Definition (Exam / Assignment Ready)

A national bank provides different types of loans to customers. Each customer belongs to one of the following loan categories:

Home Loan

Education Loan

Business Loan

The bank follows strict rules for:

Loan eligibility

Interest calculation

EMI calculation

Penalty for late repayment

You must design a multilevel inheritance system that models this behavior.

Rules & Conditions Common Rules (For All Customers)

Bank Name is same for all customers

Loan tenure is in years

EMI = Total Payable Amount / (Years × 12)

Home Loan Rules

Minimum salary: 30,000

Interest rate: 8%

Max tenure: 25 years

Education Loan Rules

Minimum marks: 60%

Interest rate: 5%

Max tenure: 10 years

Business Loan Rules

Minimum annual turnover: 5,00,000

Interest rate: 12%

Max tenure: 15 years

Penalty Rule

If tenure exceeds allowed limit → 2% extra interest added

[2]:
```
# Base Class
class Customer:
```

Vishal Acharya

```python
    bank_name = "National Secure Bank"   # class variable

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display_customer(self):
        print("Bank:", Customer.bank_name)
        print("Customer Name:", self.name)
        print("Age:", self.age)


# Intermediate Class
class Loan(Customer):
    penalty_rate = 2   # class variable (percentage)

    def __init__(self, name, age, amount, tenure):
        Customer.__init__(self, name, age)
        self.amount = amount
        self.tenure = tenure

    def calculate_total_amount(self, interest_rate):
        interest = self.amount * interest_rate * self.tenure / 100
        return self.amount + interest

    def calculate_emi(self, total_amount):
        return total_amount / (self.tenure * 12)


# Derived Class 1
class HomeLoan(Loan):
    min_salary = 30000
    interest_rate = 8
    max_tenure = 25

    def __init__(self, name, age, amount, tenure, salary):
        Loan.__init__(self, name, age, amount, tenure)
        self.salary = salary

    def process_loan(self):
        self.display_customer()
        print("Loan Type: Home Loan")

        if self.salary < HomeLoan.min_salary:
            print("Loan Rejected: Salary too low")
            return
```

Vishal Acharya

```python
        rate = HomeLoan.interest_rate
        if self.tenure > HomeLoan.max_tenure:
            rate = rate + Loan.penalty_rate

        total = self.calculate_total_amount(rate)
        emi = self.calculate_emi(total)

        print("Approved Interest Rate:", rate, "%")
        print("Total Payable Amount:", total)
        print("Monthly EMI:", emi)


# Derived Class 2
class EducationLoan(Loan):
    min_marks = 60
    interest_rate = 5
    max_tenure = 10

    def __init__(self, name, age, amount, tenure, marks):
        Loan.__init__(self, name, age, amount, tenure)
        self.marks = marks

    def process_loan(self):
        self.display_customer()
        print("Loan Type: Education Loan")

        if self.marks < EducationLoan.min_marks:
            print("Loan Rejected: Marks below eligibility")
            return

        rate = EducationLoan.interest_rate
        if self.tenure > EducationLoan.max_tenure:
            rate = rate + Loan.penalty_rate

        total = self.calculate_total_amount(rate)
        emi = self.calculate_emi(total)

        print("Approved Interest Rate:", rate, "%")
        print("Total Payable Amount:", total)
        print("Monthly EMI:", emi)


# Derived Class 3
class BusinessLoan(Loan):
    min_turnover = 500000
    interest_rate = 12
    max_tenure = 15
```

Vishal Acharya

```python
    def __init__(self, name, age, amount, tenure, turnover):
        Loan.__init__(self, name, age, amount, tenure)
        self.turnover = turnover

    def process_loan(self):
        self.display_customer()
        print("Loan Type: Business Loan")

        if self.turnover < BusinessLoan.min_turnover:
            print("Loan Rejected: Low business turnover")
            return

        rate = BusinessLoan.interest_rate
        if self.tenure > BusinessLoan.max_tenure:
            rate = rate + Loan.penalty_rate

        total = self.calculate_total_amount(rate)
        emi = self.calculate_emi(total)

        print("Approved Interest Rate:", rate, "%")
        print("Total Payable Amount:", total)
        print("Monthly EMI:", emi)


# ---------- Object Creation ----------
print("----- HOME LOAN -----")
h = HomeLoan("Amit", 35, 2000000, 28, 42000)
h.process_loan()

print("\n----- EDUCATION LOAN -----")
e = EducationLoan("Riya", 21, 800000, 8, 72)
e.process_loan()

print("\n----- BUSINESS LOAN -----")
b = BusinessLoan("Karan", 40, 3000000, 18, 450000)
b.process_loan()
```

```
----- HOME LOAN -----
Bank: National Secure Bank
Customer Name: Amit
Age: 35
Loan Type: Home Loan
Approved Interest Rate: 10 %
Total Payable Amount: 7600000.0
Monthly EMI: 22619.04761904762

----- EDUCATION LOAN -----
```

Vishal Acharya

```
Bank: National Secure Bank
Customer Name: Riya
Age: 21
Loan Type: Education Loan
Approved Interest Rate: 5 %
Total Payable Amount: 1120000.0
Monthly EMI: 11666.666666666666

----- BUSINESS LOAN -----
Bank: National Secure Bank
Customer Name: Karan
Age: 40
Loan Type: Business Loan
Loan Rejected: Low business turnover
```

5 A national smart power grid supplies electricity to different types of consumers.

Each consumer type follows different billing, subsidy, and penalty rules, but the billing interface must be common.

You must design an inheritance hierarchy where the same method name behaves differently at runtime (polymorphism).

Consumer Types

DomesticConsumer

CommercialConsumer

IndustrialConsumer

Common Rules (All Consumers)

Electricity unit = kWh

Billing cycle = monthly

Grid surcharge applies if consumption exceeds grid limit

All consumers must implement calculate_bill()

Final bill = Base Bill − Subsidy + Penalty

Class Variable Rules

Parameter Value

Grid Name "National Smart Grid"

Grid Limit 500 units

Surcharge Rate 2 per excess unit

Consumer-Specific Rules

Vishal Acharya

Domestic Consumer

Rate: 4 per unit

Subsidy: 500 if usage 200 units

Penalty: 100 if usage > 500 units

Commercial Consumer

Rate: 7 per unit

No subsidy

Penalty: 5 per excess unit above 500

Industrial Consumer

Rate: 10 per unit

Demand charge: 2000 fixed

Penalty: 10 per excess unit above 500

Polymorphism Requirement (MANDATORY)

A single loop must call calculate_bill() on different consumer objects, and each object must behave differently without checking its type.

[3]:
```
# Base Class
class Consumer:
    grid_name = "National Smart Grid"     # class variable
    grid_limit = 500                      # class variable
    surcharge_rate = 2                   # class variable

    def __init__(self, name, units):
        self.name = name
        self.units = units

    def calculate_bill(self):
        print("Base consumer billing - not applicable")

    def grid_surcharge(self):
        if self.units > Consumer.grid_limit:
            return (self.units - Consumer.grid_limit) * Consumer.surcharge_rate
        return 0

    def display_common(self):
        print("Grid:", Consumer.grid_name)
        print("Consumer Name:", self.name)
        print("Units Consumed:", self.units)


# Derived Class 1
```

Vishal Acharya

```python
class DomesticConsumer(Consumer):
    rate = 4
    subsidy_limit = 200
    subsidy_amount = 500
    penalty_amount = 100

    def calculate_bill(self):
        self.display_common()
        print("Consumer Type: Domestic")

        base_bill = self.units * DomesticConsumer.rate
        subsidy = 0
        penalty = 0

        if self.units <= DomesticConsumer.subsidy_limit:
            subsidy = DomesticConsumer.subsidy_amount

        if self.units > Consumer.grid_limit:
            penalty = DomesticConsumer.penalty_amount

        total = base_bill - subsidy + penalty + self.grid_surcharge()

        print("Base Bill:", base_bill)
        print("Subsidy:", subsidy)
        print("Penalty:", penalty)
        print("Grid Surcharge:", self.grid_surcharge())
        print("Final Bill:", total)


# Derived Class 2
class CommercialConsumer(Consumer):
    rate = 7
    excess_penalty_rate = 5

    def calculate_bill(self):
        self.display_common()
        print("Consumer Type: Commercial")

        base_bill = self.units * CommercialConsumer.rate
        penalty = 0

        if self.units > Consumer.grid_limit:
            penalty = (self.units - Consumer.grid_limit) * CommercialConsumer.
↪excess_penalty_rate

        total = base_bill + penalty + self.grid_surcharge()
```

Vishal Acharya

```python
        print("Base Bill:", base_bill)
        print("Penalty:", penalty)
        print("Grid Surcharge:", self.grid_surcharge())
        print("Final Bill:", total)


# Derived Class 3
class IndustrialConsumer(Consumer):
    rate = 10
    demand_charge = 2000
    excess_penalty_rate = 10

    def calculate_bill(self):
        self.display_common()
        print("Consumer Type: Industrial")

        base_bill = self.units * IndustrialConsumer.rate
        penalty = 0

        if self.units > Consumer.grid_limit:
            penalty = (self.units - Consumer.grid_limit) * IndustrialConsumer.
 ↪excess_penalty_rate

        total = base_bill + IndustrialConsumer.demand_charge + penalty + self.
 ↪grid_surcharge()

        print("Base Bill:", base_bill)
        print("Demand Charge:", IndustrialConsumer.demand_charge)
        print("Penalty:", penalty)
        print("Grid Surcharge:", self.grid_surcharge())
        print("Final Bill:", total)


# -------- POLYMORPHISM IN ACTION --------
consumers = [
    DomesticConsumer("Amit", 180),
    CommercialConsumer("Reliance Store", 620),
    IndustrialConsumer("Steel Plant", 850)
]

for c in consumers:           # SAME METHOD CALL
    print("\n----------------------------")
    c.calculate_bill()        # DIFFERENT BEHAVIOR (Runtime Polymorphism)
```

```
----------------------------
Grid: National Smart Grid
```

Vishal Acharya

```
Consumer Name: Amit
Units Consumed: 180
Consumer Type: Domestic
Base Bill: 720
Subsidy: 500
Penalty: 0
Grid Surcharge: 0
Final Bill: 220

----------------------------
Grid: National Smart Grid
Consumer Name: Reliance Store
Units Consumed: 620
Consumer Type: Commercial
Base Bill: 4340
Penalty: 600
Grid Surcharge: 240
Final Bill: 5180

----------------------------
Grid: National Smart Grid
Consumer Name: Steel Plant
Units Consumed: 850
Consumer Type: Industrial
Base Bill: 8500
Demand Charge: 2000
Penalty: 3500
Grid Surcharge: 700
Final Bill: 14700
```

6 A national electricity board monitors daily power consumption (units) of 4 cities across 8 days.

The data is provided as a 2D NumPy array:

Rows → Days

Columns → Cities

Each value represents units consumed per city per day.

INPUT DATA (Fixed) import numpy as np

data = np.array([[120, 340, 560, 230], [150, 360, 590, 260], [180, 390, 610, 290], [200, 410, 640, 310], [220, 430, 660, 330], [250, 470, 700, 360], [280, 500, 730, 390], [300, 540, 780, 420]])

TASKS (ALL MANDATORY)

Task 1 — Shape & Structural Validation

Print array shape

Vishal Acharya

Reshape data into 2 blocks of 4 days using array_split

Task 2 — City-wise Violation Detection

A city is violating on a day if:

Consumption > 600 units

Detect all violations using where

Store results in a dictionary:

{ city_index : [list of violating days] }

Task 3 — Critical Cities Identification

A city becomes critical if it violates on 3 or more days

Store critical cities in a set

Task 4 — Daily High-Load City Extraction

For each day:

Identify cities with consumption > 500

Store as:

day_number : [city indices]

Task 5 — Sorted Violation Matrix

Extract only violating values (>600)

Sort them using sort

Convert result into a Python list

Task 6 — Flattening via Iteration

Iterate manually (no flatten())

Create a list of all values

Remove duplicates using set

Convert back to sorted list

Task 7 — City Growth Analysis

Compare Day 1 vs Day 8

Calculate growth using indexing only

Store result in dictionary:

{ city_index : growth_value }

[4]:
```
import numpy as np

data = np.array([
```

Vishal Acharya

```
    [120, 340, 560, 230],
    [150, 360, 590, 260],
    [180, 390, 610, 290],
    [200, 410, 640, 310],
    [220, 430, 660, 330],
    [250, 470, 700, 360],
    [280, 500, 730, 390],
    [300, 540, 780, 420]
])

# Task 1
print("Shape:", data.shape)
blocks = np.array_split(data, 2)
print("Blocks:", blocks)

# Task 2
violations = {}
rows, cols = data.shape

for c in range(cols):
    days = []
    for r in range(rows):
        if data[r][c] > 600:
            days.append(r)
    if days:
        violations[c] = days

print("Violations:", violations)

# Task 3
critical_cities = set()
for city in violations:
    if len(violations[city]) >= 3:
        critical_cities.add(city)

print("Critical Cities:", critical_cities)

# Task 4
daily_high_load = {}

for r in range(rows):
    cities = []
    for c in range(cols):
        if data[r][c] > 500:
            cities.append(c)
    if cities:
        daily_high_load[r] = cities
```

Vishal Acharya

```python
print("Daily High Load:", daily_high_load)

# Task 5
violation_values = []
positions = np.where(data > 600)

for i in range(len(positions[0])):
    violation_values.append(data[positions[0][i]][positions[1][i]])

sorted_violations = np.sort(np.array(violation_values))
print("Sorted Violations:", list(sorted_violations))

# Task 6
all_values = []

for r in range(rows):
    for c in range(cols):
        all_values.append(data[r][c])

unique_sorted = sorted(list(set(all_values)))
print("Unique Sorted Values:", unique_sorted)

# Task 7
growth = {}

for c in range(cols):
    growth[c] = data[7][c] - data[0][c]

print("City Growth:", growth)
```

```
Shape: (8, 4)
Blocks: [array([[120, 340, 560, 230],
       [150, 360, 590, 260],
       [180, 390, 610, 290],
       [200, 410, 640, 310]]), array([[220, 430, 660, 330],
       [250, 470, 700, 360],
       [280, 500, 730, 390],
       [300, 540, 780, 420]])]
Violations: {2: [2, 3, 4, 5, 6, 7]}
Critical Cities: {2}
Daily High Load: {0: [2], 1: [2], 2: [2], 3: [2], 4: [2], 5: [2], 6: [2], 7: [1,
2]}
Sorted Violations: [610, 640, 660, 700, 730, 780]
Unique Sorted Values: [120, 150, 180, 200, 220, 230, 250, 260, 280, 290, 300,
310, 330, 340, 360, 390, 410, 420, 430, 470, 500, 540, 560, 590, 610, 640, 660,
700, 730, 780]
City Growth: {0: 180, 1: 200, 2: 220, 3: 190}
```

Vishal Acharya

7 Solar + Grid hybrid violation analytics

University 90-minute challenge paper

GIVEN DATA (DO NOT MODIFY) import numpy as np

solar = np.array([[120, 180, 90, 150, 200], [130, 190, 100, 160, 210], [140, 200, 110, 170, 220], [150, 210, 120, 180, 230], [160, 220, 130, 190, 240], [170, 230, 140, 200, 250], [180, 240, 150, 210, 260], [190, 250, 160, 220, 270], [200, 260, 170, 230, 280], [210, 270, 180, 240, 290]])

grid = np.array([[300, 420, 380, 410, 390], [320, 430, 400, 420, 410], [340, 450, 420, 440, 430], [360, 470, 440, 460, 450], [380, 490, 460, 480, 470], [400, 510, 480, 500, 490], [420, 530, 500, 520, 510], [440, 550, 520, 540, 530], [460, 570, 540, 560, 550], [480, 590, 560, 580, 570]])

Rows → Days (0 to 9)

Columns → City sectors (0 to 4)

TASKS Q1 (3 Marks) — Structural Validation

Print the shape of both arrays

Split both solar and grid into two equal blocks of 5 days using array_split

Q2 (5 Marks) — Hybrid Load Calculation

Using manual iteration only:

Create a new 2D NumPy array total_energy

Each value = solar + grid (same day, same sector)

Q3 (6 Marks) — Grid Dependency Violation Detection

A Grid Dependency Violation occurs if:

grid_units > solar_units × 2

Detect all violations using where

Store results in a dictionary:

{ sector_index : [list of violating days] }

Q4 (5 Marks) — Critical Sector Identification

A sector is CRITICAL if:

It violates on 4 or more days

Store all critical sectors in a set.

Q5 (4 Marks) — Day-wise Grid Stress Mapping

For each day:

Identify sectors where grid usage > 500

Store result in a dictionary:

day_index : [sector indices]

Vishal Acharya

Q6 (4 Marks) — Violation Severity Analysis

Extract only grid values where violations occurred

Sort them using NumPy sort

Convert final output to a Python list

Q7 (3 Marks) — Solar Growth Audit

Compare Day 0 vs Day 9 solar production

Store growth per sector in a dictionary:

{ sector_index : growth_units }

[5]:
```
import numpy as np

# ---------------- GIVEN DATA ----------------
solar = np.array([
    [120, 180, 90, 150, 200],
    [130, 190, 100, 160, 210],
    [140, 200, 110, 170, 220],
    [150, 210, 120, 180, 230],
    [160, 220, 130, 190, 240],
    [170, 230, 140, 200, 250],
    [180, 240, 150, 210, 260],
    [190, 250, 160, 220, 270],
    [200, 260, 170, 230, 280],
    [210, 270, 180, 240, 290]
])

grid = np.array([
    [300, 420, 380, 410, 390],
    [320, 430, 400, 420, 410],
    [340, 450, 420, 440, 430],
    [360, 470, 440, 460, 450],
    [380, 490, 460, 480, 470],
    [400, 510, 480, 500, 490],
    [420, 530, 500, 520, 510],
    [440, 550, 520, 540, 530],
    [460, 570, 540, 560, 550],
    [480, 590, 560, 580, 570]
])

# ---------------- Q1: Shape & Split ----------------
print("Solar Shape:", solar.shape)
print("Grid Shape:", grid.shape)

solar_blocks = np.array_split(solar, 2)
grid_blocks = np.array_split(grid, 2)
```

Vishal Acharya

```python
print("Solar Blocks:", solar_blocks)
print("Grid Blocks:", grid_blocks)

# ---------------- Q2: Total Hybrid Energy ----------------
rows, cols = solar.shape
total_energy_list = []

for r in range(rows):
    row_data = []
    for c in range(cols):
        row_data.append(solar[r][c] + grid[r][c])
    total_energy_list.append(row_data)

total_energy = np.array(total_energy_list)
print("Total Hybrid Energy:\n", total_energy)

# ---------------- Q3: Grid Dependency Violations ----------------
violations = {}

positions = np.where(grid > solar * 2)

for i in range(len(positions[0])):
    day = positions[0][i]
    sector = positions[1][i]

    if sector not in violations:
        violations[sector] = []
    violations[sector].append(day)

print("Grid Dependency Violations:", violations)

# ---------------- Q4: Critical Sectors ----------------
critical_sectors = set()

for sector in violations:
    if len(violations[sector]) >= 4:
        critical_sectors.add(sector)

print("Critical Sectors:", critical_sectors)

# ---------------- Q5: Day-wise Grid Stress ----------------
grid_stress = {}

for r in range(rows):
    stressed_sectors = []
    for c in range(cols):
```

Vishal Acharya

```python
        if grid[r][c] > 500:
            stressed_sectors.append(c)
    if stressed_sectors:
        grid_stress[r] = stressed_sectors

print("Day-wise Grid Stress:", grid_stress)

# ---------------- Q6: Violation Severity ----------------
violation_values = []

for i in range(len(positions[0])):
    violation_values.append(grid[positions[0][i]][positions[1][i]])

sorted_violations = np.sort(np.array(violation_values))
print("Sorted Violation Severity:", list(sorted_violations))

# ---------------- Q7: Solar Growth Audit ----------------
solar_growth = {}

for c in range(cols):
    solar_growth[c] = solar[9][c] - solar[0][c]

print("Solar Growth per Sector:", solar_growth)
```

```
Solar Shape: (10, 5)
Grid Shape: (10, 5)
Solar Blocks: [array([[120, 180,  90, 150, 200],
       [130, 190, 100, 160, 210],
       [140, 200, 110, 170, 220],
       [150, 210, 120, 180, 230],
       [160, 220, 130, 190, 240]]), array([[170, 230, 140, 200, 250],
       [180, 240, 150, 210, 260],
       [190, 250, 160, 220, 270],
       [200, 260, 170, 230, 280],
       [210, 270, 180, 240, 290]])]
Grid Blocks: [array([[300, 420, 380, 410, 390],
       [320, 430, 400, 420, 410],
       [340, 450, 420, 440, 430],
       [360, 470, 440, 460, 450],
       [380, 490, 460, 480, 470]]), array([[400, 510, 480, 500, 490],
       [420, 530, 500, 520, 510],
       [440, 550, 520, 540, 530],
       [460, 570, 540, 560, 550],
       [480, 590, 560, 580, 570]])]
Total Hybrid Energy:
 [[420 600 470 560 590]
 [450 620 500 580 620]
 [480 650 530 610 650]
```

Vishal Acharya

```
 [510 680 560 640 680]
 [540 710 590 670 710]
 [570 740 620 700 740]
 [600 770 650 730 770]
 [630 800 680 760 800]
 [660 830 710 790 830]
 [690 860 740 820 860]]
Grid Dependency Violations: {0: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9], 1: [0, 1, 2, 3,
4, 5, 6, 7, 8, 9], 2: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9], 3: [0, 1, 2, 3, 4, 5, 6,
7, 8, 9]}
Critical Sectors: {0, 1, 2, 3}
Day-wise Grid Stress: {5: [1], 6: [1, 3, 4], 7: [1, 2, 3, 4], 8: [1, 2, 3, 4],
9: [1, 2, 3, 4]}
Sorted Violation Severity: [300, 320, 340, 360, 380, 380, 400, 400, 410, 420,
420, 420, 420, 430, 440, 440, 440, 450, 460, 460, 460, 470, 480, 480, 480, 490,
500, 500, 510, 520, 520, 530, 540, 540, 550, 560, 560, 570, 580, 590]
Solar Growth per Sector: {0: 90, 1: 90, 2: 90, 3: 90, 4: 90}
```

8 PROBLEM STATEMENT Retail Sales Performance & Violation Analytics System

A retail company operates three types of stores:

Local Store

Online Store

Wholesale Store

Sales data for 5 products over 7 days is collected.

Each store type follows different rules for:

Revenue calculation

Discount

Violation detection

You must design an OOP-based system where:

A common interface is used

Each store type behaves differently at runtime (polymorphism)

Sales analysis is displayed using Streamlit

Calculations use NumPy

sales = np.array([[1200, 900, 1500, 1100, 1700], [1300, 950, 1600, 1200, 1800], [1250, 980, 1700, 1250, 1900], [1400, 1000, 1800, 1300, 2000], [1500, 1050, 1900, 1350, 2100], [1600, 1100, 2000, 1400, 2200], [1700, 1200, 2100, 1500, 2300]])

Rows → Days (0–6)

Vishal Acharya

Columns → Products (0–4)

Values → Sales amount ( )

BUSINESS RULES

Local Store

Discount: 200 if daily total < 6000

Violation: If any product sale < 1000

Online Store

Discount: 5% if daily total > 7000

Violation: If daily total < 5000

Wholesale Store

Discount: Flat 500

Violation: If any product sale < 1500

TASKS

1. Display sales matrix & shape
2. Calculate day-wise total sales (manual iteration)
3. Apply store-specific discount logic
4. Detect violations using np.where
5. Identify critical days (2 violations)
6. Sort violation values
7. Display everything interactively in Streamlit

[6]:
```
%%writefile sales_app.py
import streamlit as st
import numpy as np

# ---------------- DATA ----------------
sales = np.array([
    [1200, 900, 1500, 1100, 1700],
    [1300, 950, 1600, 1200, 1800],
    [1250, 980, 1700, 1250, 1900],
    [1400, 1000, 1800, 1300, 2000],
    [1500, 1050, 1900, 1350, 2100],
    [1600, 1100, 2000, 1400, 2200],
    [1700, 1200, 2100, 1500, 2300]
])

# ---------------- BASE CLASS ----------------
class Store:
```

Vishal Acharya

```python
    company = "Smart Retail Pvt Ltd"

    def __init__(self, sales):
        self.sales = sales

    def analyze_sales(self):
        pass

    def daily_totals(self):
        totals = []
        for r in range(self.sales.shape[0]):
            s = 0
            for c in range(self.sales.shape[1]):
                s += self.sales[r][c]
            totals.append(s)
        return totals


# ---------------- DERIVED CLASSES ----------------
class LocalStore(Store):
    def analyze_sales(self):
        totals = self.daily_totals()
        violations = np.where(self.sales < 1000)

        st.subheader("  Local Store Analysis")
        st.write("Daily Totals:", totals)
        st.write("Violations (sales < 1000):", list(zip(violations[0],␣
 ↪violations[1])))


class OnlineStore(Store):
    def analyze_sales(self):
        totals = self.daily_totals()
        low_days = []

        for i, t in enumerate(totals):
            if t < 5000:
                low_days.append(i)

        st.subheader("  Online Store Analysis")
        st.write("Daily Totals:", totals)
        st.write("Low Revenue Days:", low_days)


class WholesaleStore(Store):
    def analyze_sales(self):
        violations = np.where(self.sales < 1500)
```

Vishal Acharya

```
        values = []

        for i in range(len(violations[0])):
            values.append(self.sales[violations[0][i]][violations[1][i]])

        sorted_vals = np.sort(np.array(values))

        st.subheader("  Wholesale Store Analysis")
        st.write("Violation Values Sorted:", list(sorted_vals))


# ---------------- STREAMLIT UI ----------------
st.title("  Sales Analytics using OOP + NumPy")

st.write("Company:", Store.company)
st.write("Sales Data Matrix:")
st.write(sales)
st.write("Shape:", sales.shape)

store_type = st.selectbox(
    "Select Store Type",
    ["Local Store", "Online Store", "Wholesale Store"]
)

if st.button("Analyze Sales"):
    if store_type == "Local Store":
        store = LocalStore(sales)
    elif store_type == "Online Store":
        store = OnlineStore(sales)
    else:
        store = WholesaleStore(sales)

    #   POLYMORPHISM
    store.analyze_sales()
```

```
Writing sales_app.py
```

9 A manufacturing company inspects 6 production batches using 5 quality parameters. Each product category has different quality rules.

Product Types

Electronics

Mechanical

Chemical

Vishal Acharya

quality_data = np.array([[78, 85, 92, 88, 90], [82, 80, 89, 86, 91], [75, 88, 90, 84, 87], [90, 92, 94, 91, 93], [68, 72, 80, 75, 78], [88, 90, 91, 89, 92]])

TASKS COVERED (ALL 17)

1. Shape validation
2. Batch-wise defect detection
3. Defect count classification
4. Critical batch identification
5. Manual average calculation
6. Parameter stability analysis
7. Violation severity extraction & sorting
8. Final quality decision report
9. Runtime polymorphism demonstration
10. Batch-wise quality trend graph
11. Parameter failure frequency graph
12. Defect severity distribution graph
13. Product-wise critical batch comparison
14. Violation severity trend graph
15. Accept / Rework / Reject distribution graph
16. Polymorphic graph generation
17. Quality degradation forecast graph

[11]:
```
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import Image, display

# ---------------- BASE CLASS ----------------
class QualityInspection:
    company = "Advanced Manufacturing Ltd"

    def __init__(self, data):
        self.data = data

    def analyze_quality(self):
        pass

    def generate_graph(self):
        pass
```

Vishal Acharya

```python
    def batch_averages(self):
        avgs = []
        for r in range(self.data.shape[0]):
            total = 0
            for c in range(self.data.shape[1]):
                total += self.data[r][c]
            avgs.append(total / self.data.shape[1])
        return avgs


# ---------------- ELECTRONICS ----------------
class Electronics(QualityInspection):
    def analyze_quality(self):
        defects, critical, decisions = {}, set(), {}

        for b in range(self.data.shape[0]):
            bad = []
            for p in range(self.data.shape[1]):
                if self.data[b][p] < 80:
                    bad.append(p)

            if bad:
                defects[b] = bad

            if len(bad) >= 2:
                critical.add(b)
                decisions[b] = "REWORK"
            else:
                decisions[b] = "ACCEPT"

        print("\n Electronics Analysis")
        print("Defects:", defects)
        print("Critical Batches:", critical)
        print("Decisions:", decisions)

        return defects, critical, decisions

    def generate_graph(self):
        plt.plot(self.batch_averages())
        plt.xlabel("Batch")
        plt.ylabel("Average Quality")
        plt.title("Electronics Batch Quality Trend")
        plt.savefig("electronics_quality_trend.png")
        plt.close()
        display(Image("electronics_quality_trend.png"))
```

Vishal Acharya

```python
# ---------------- MECHANICAL ----------------
class Mechanical(QualityInspection):
    def analyze_quality(self):
        defects, critical, decisions = {}, set(), {}

        for b in range(self.data.shape[0]):
            total = 0
            bad = []
            for p in range(self.data.shape[1]):
                total += self.data[b][p]
                if self.data[b][p] < 75:
                    bad.append(p)

            avg = total / self.data.shape[1]

            if bad:
                defects[b] = bad

            if avg < 85:
                critical.add(b)
                decisions[b] = "REWORK"
            else:
                decisions[b] = "ACCEPT"

        print("\n Mechanical Analysis")
        print("Defects:", defects)
        print("Critical Batches:", critical)
        print("Decisions:", decisions)

        return defects, critical, decisions

    def generate_graph(self):
        failures = {}

        for p in range(self.data.shape[1]):
            count = 0
            for b in range(self.data.shape[0]):
                if self.data[b][p] < 75:
                    count += 1
            failures[p] = count

        plt.bar(failures.keys(), failures.values())
        plt.xlabel("Parameter")
        plt.ylabel("Failure Count")
        plt.title("Mechanical Parameter Failures")
        plt.savefig("mechanical_parameter_failures.png")
        plt.close()
```

Vishal Acharya

```
        display(Image("mechanical_parameter_failures.png"))


# ---------------- CHEMICAL ----------------
class Chemical(QualityInspection):
    def analyze_quality(self):
        defects, critical, decisions = {}, set(), {}

        for b in range(self.data.shape[0]):
            bad = []
            for p in range(self.data.shape[1]):
                if self.data[b][p] < 85:
                    bad.append(p)

            if bad:
                defects[b] = bad

            if len(bad) >= 3:
                critical.add(b)
                decisions[b] = "REJECT"
            else:
                decisions[b] = "ACCEPT"

        print("\n Chemical Analysis")
        print("Defects:", defects)
        print("Critical Batches:", critical)
        print("Decisions:", decisions)

        return defects, critical, decisions

    def generate_graph(self):
        pos = np.where(self.data < 85)
        vals = []

        for i in range(len(pos[0])):
            vals.append(self.data[pos[0][i]][pos[1][i]])

        sorted_vals = np.sort(np.array(vals))
        plt.plot(sorted_vals)
        plt.xlabel("Violation Index")
        plt.ylabel("Severity")
        plt.title("Chemical Violation Severity Trend")
        plt.savefig("chemical_violation_severity.png")
        plt.close()
        display(Image("chemical_violation_severity.png"))
```

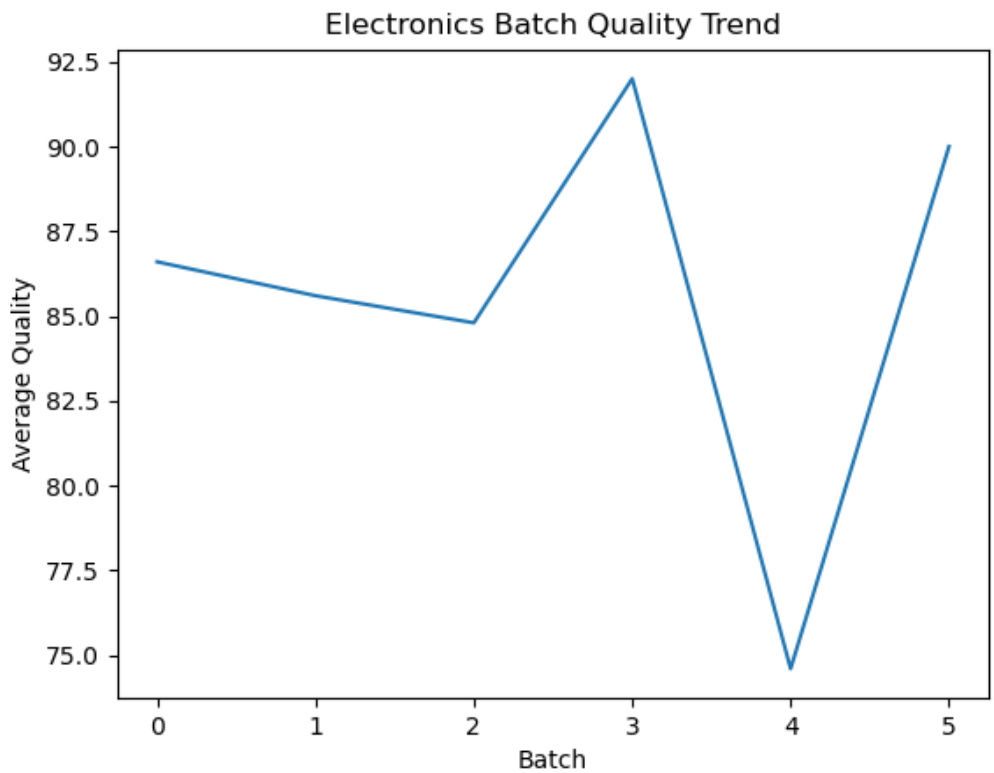
Vishal Acharya

```python
# ---------------- MAIN EXECUTION ----------------
quality_data = np.array([
    [78, 85, 92, 88, 90],
    [82, 80, 89, 86, 91],
    [75, 88, 90, 84, 87],
    [90, 92, 94, 91, 93],
    [68, 72, 80, 75, 78],
    [88, 90, 91, 89, 92]
])

print("Company:", QualityInspection.company)
print("Quality Data Shape:", quality_data.shape)

objects = [
    Electronics(quality_data),
    Mechanical(quality_data),
    Chemical(quality_data)
]

#   POLYMORPHISM
for obj in objects:
    obj.analyze_quality()
    obj.generate_graph()


# ---------------- TASK 17: QUALITY DEGRADATION FORECAST ----------------
print("\n  Quality Degradation Forecast")
degraded = {}

for p in range(quality_data.shape[1]):
    diff = quality_data[-1][p] - quality_data[0][p]
    if diff < 0:
        degraded[p] = diff

if degraded:
    plt.bar(degraded.keys(), degraded.values())
    plt.xlabel("Parameter")
    plt.ylabel("Degradation")
    plt.title("Quality Degradation Forecast")
    plt.savefig("quality_degradation_forecast.png")
    plt.close()
    display(Image("quality_degradation_forecast.png"))
else:
    print("No degradation detected")
```

```
Company: Advanced Manufacturing Ltd
Quality Data Shape: (6, 5)
```

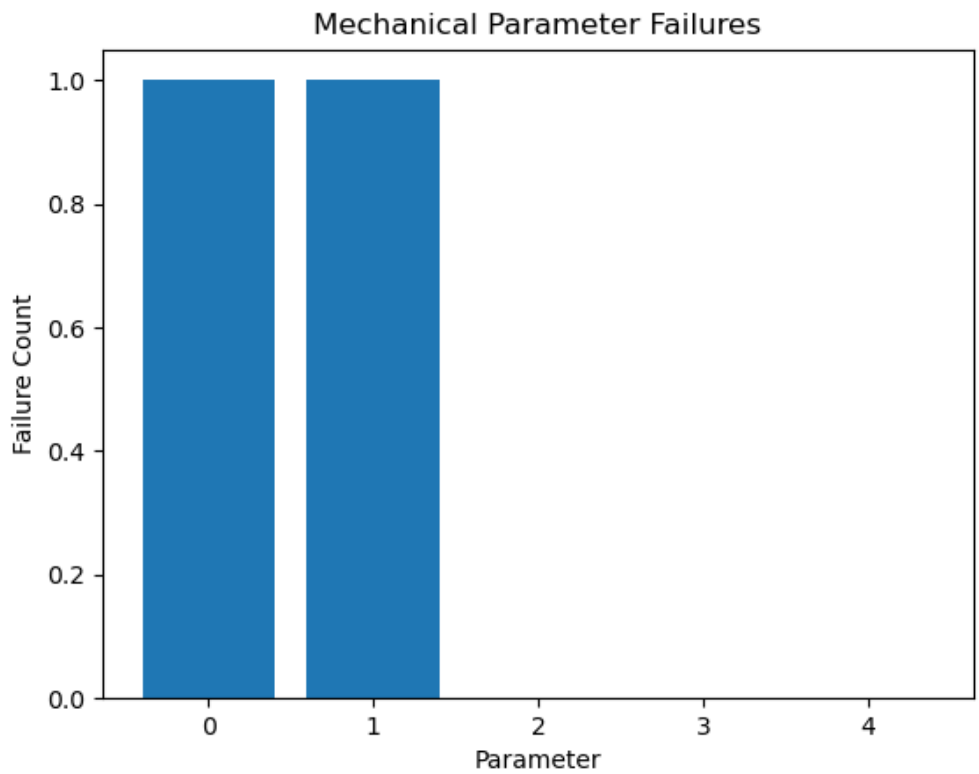
Vishal Acharya

```
 Electronics Analysis
Defects: {0: [0], 2: [0], 4: [0, 1, 3, 4]}
Critical Batches: {4}
Decisions: {0: 'ACCEPT', 1: 'ACCEPT', 2: 'ACCEPT', 3: 'ACCEPT', 4: 'REWORK', 5:
'ACCEPT'}
```



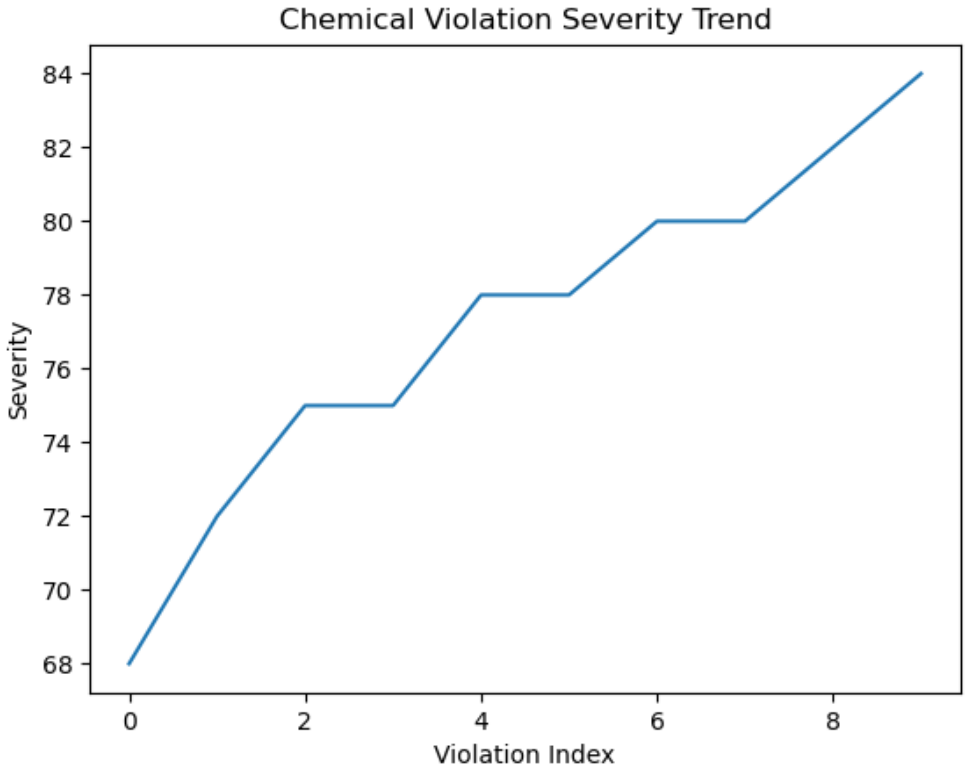

```
 Mechanical Analysis
Defects: {4: [0, 1]}
Critical Batches: {2, 4}
Decisions: {0: 'ACCEPT', 1: 'ACCEPT', 2: 'REWORK', 3: 'ACCEPT', 4: 'REWORK', 5:
'ACCEPT'}
```

Vishal Acharya



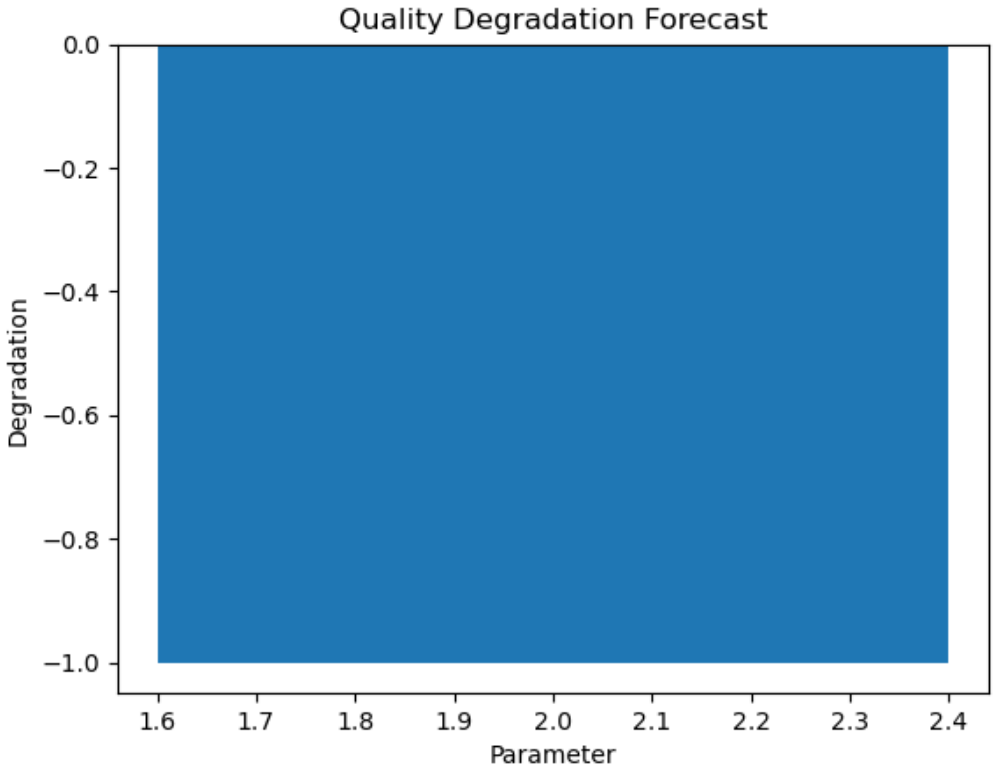

```
 Chemical Analysis
Defects: {0: [0], 1: [0, 1], 2: [0, 3], 4: [0, 1, 2, 3, 4]}
Critical Batches: {4}
Decisions: {0: 'ACCEPT', 1: 'ACCEPT', 2: 'ACCEPT', 3: 'ACCEPT', 4: 'REJECT', 5:
'ACCEPT'}
```

Vishal Acharya




```
Quality Degradation Forecast
```

Vishal Acharya



[8]:
```
%%writefile quality_data.py
import streamlit as st
import numpy as np
import matplotlib.pyplot as plt

# ---------------- DATA ----------------
quality_data = np.array([
    [78, 85, 92, 88, 90],
    [82, 80, 89, 86, 91],
    [75, 88, 90, 84, 87],
    [90, 92, 94, 91, 93],
    [68, 72, 80, 75, 78],
    [88, 90, 91, 89, 92]
])

# ---------------- BASE CLASS ----------------
class QualityInspection:
    company = "Advanced Manufacturing Ltd"

    def __init__(self, data):
```

Vishal Acharya

```python
        self.data = data

    def analyze_quality(self):
        pass

    def generate_graph(self):
        pass

    def batch_averages(self):
        avgs = []
        for r in range(self.data.shape[0]):
            total = 0
            for c in range(self.data.shape[1]):
                total += self.data[r][c]
            avgs.append(total / self.data.shape[1])
        return avgs


# ---------------- ELECTRONICS ----------------
class Electronics(QualityInspection):
    def analyze_quality(self):
        defects, critical, decisions = {}, set(), {}
        for b in range(self.data.shape[0]):
            bad = []
            for p in range(self.data.shape[1]):
                if self.data[b][p] < 80:
                    bad.append(p)
            if bad:
                defects[b] = bad
            if len(bad) >= 2:
                critical.add(b)
                decisions[b] = "REWORK"
            else:
                decisions[b] = "ACCEPT"
        return defects, critical, decisions

    def generate_graph(self):
        st.subheader("  Electronics - Batch Quality Trend")
        plt.plot(self.batch_averages())
        st.pyplot(plt)


# ---------------- MECHANICAL ----------------
class Mechanical(QualityInspection):
    def analyze_quality(self):
        defects, critical, decisions = {}, set(), {}
        for b in range(self.data.shape[0]):
```

Vishal Acharya

```
            total, bad = 0, []
            for p in range(self.data.shape[1]):
                total += self.data[b][p]
                if self.data[b][p] < 75:
                    bad.append(p)
            avg = total / self.data.shape[1]
            if bad:
                defects[b] = bad
            if avg < 85:
                critical.add(b)
                decisions[b] = "REWORK"
            else:
                decisions[b] = "ACCEPT"
        return defects, critical, decisions

    def generate_graph(self):
        st.subheader("  Mechanical - Parameter Failure Count")
        counts = {}
        for p in range(self.data.shape[1]):
            count = 0
            for b in range(self.data.shape[0]):
                if self.data[b][p] < 75:
                    count += 1
            counts[p] = count
        plt.bar(counts.keys(), counts.values())
        st.pyplot(plt)


# ---------------- CHEMICAL ----------------
class Chemical(QualityInspection):
    def analyze_quality(self):
        defects, critical, decisions = {}, set(), {}
        for b in range(self.data.shape[0]):
            bad = []
            for p in range(self.data.shape[1]):
                if self.data[b][p] < 85:
                    bad.append(p)
            if bad:
                defects[b] = bad
            if len(bad) >= 3:
                critical.add(b)
                decisions[b] = "REJECT"
            else:
                decisions[b] = "ACCEPT"
        return defects, critical, decisions

    def generate_graph(self):
```

Vishal Acharya

```
        st.subheader("  Chemical - Violation Severity Trend")
        pos = np.where(self.data < 85)
        vals = []
        for i in range(len(pos[0])):
            vals.append(self.data[pos[0][i]][pos[1][i]])
        sorted_vals = np.sort(np.array(vals))
        plt.plot(sorted_vals)
        st.pyplot(plt)


# ---------------- STREAMLIT UI ----------------
st.title("  Quality Analysis System (Ultra-Hard)")

st.write("Company:", QualityInspection.company)
st.write("Quality Matrix:")
st.write(quality_data)
st.write("Shape:", quality_data.shape)

choice = st.selectbox(
    "Select Product Type",
    ["Electronics", "Mechanical", "Chemical"]
)

if st.button("Run Complete Analysis"):
    if choice == "Electronics":
        obj = Electronics(quality_data)
    elif choice == "Mechanical":
        obj = Mechanical(quality_data)
    else:
        obj = Chemical(quality_data)

    #   POLYMORPHISM
    defects, critical, decisions = obj.analyze_quality()

    st.subheader("Defect Dictionary")
    st.write(defects)

    st.subheader("Critical Batches")
    st.write(critical)

    st.subheader("Final Decisions")
    st.write(decisions)

    obj.generate_graph()

    # Degradation Forecast (Task 17)
    st.subheader("  Quality Degradation Forecast")
```

Vishal Acharya

```
    degraded = {}
    for p in range(quality_data.shape[1]):
        diff = quality_data[-1][p] - quality_data[0][p]
        if diff < 0:
            degraded[p] = diff

    if degraded:
        plt.bar(degraded.keys(), degraded.values())
        st.pyplot(plt)
    else:
        st.write("No degradation detected")
```

```
Writing quality_data.py
```

[ ]:

```

```

Vishal Acharya